QuietLetter - technical report

Track 2B - Own Project | Online | Jacek Adamek (solo)

Started 14 October 2026. Development report, not a submitted entry. Demo video: not recorded. Public source published; clean-checkout Docker verification passed on 14 October 2026.

1. Summary

QuietLetter helps a reader review short everyday correspondence: requests, literal dates and explicit uncertainty. Apertus selects passage IDs instead of writing a free-form summary. The app resolves each selected ID to an exact source quote and character offsets. This verifies text provenance, not classification correctness or completeness. The reader keeps the original and decides what to do. No legal advice, automatic reply, payment, calendar update or external action is performed.

2. Architecture and sovereign deployment

Browser -> same-origin Node HTTP app -> authenticated loopback llama.cpp -> local Apertus weights -> bounded JSON selection -> source validator -> review UI. All dynamic text is rendered using DOM textContent/value, never HTML. Letters remain in session/request memory, not persistent storage. The reader can open and intentionally copy a read-only JSON report. Editing the text invalidates the old result.

The target is on-premise deployment under the operator's administration. The Docker recipe runs the app and model in one container with weights mounted read only. Network is used during initial download and image build. Application code makes no external runtime requests, but Docker's default network is not an enforced air gap. No audited air-gap claim is made.

Native Windows binds to loopback; container deployment publishes only a loopback host port. The model has a random API secret generated in memory for each session. An actual unauthenticated model request returned HTTP 401. Foreign origins and Host headers are rejected by the web app. Letters and model prompts are not printed to logs.

Inputs are limited to 6000 UTF-16 code units and 48 passages. Responses are limited to 64 KB, 160 generated tokens and 180 seconds. One analysis runs at a time. Malformed, incomplete or unsupported selections fail closed. Missing/failing Apertus returns an error; no paid API or substitute model is used. A small date scanner returns literal expressions without calculating due dates.

3. Use of Apertus

Model: Colby/apertus-v1.5-8b-text-Q4_K_M-GGUF, a community text-only quantization derived from andreasmartin/apertus-v1.5-8b-text and swiss-ai/Apertus-v1.5-8B. License: Apache-2.0. This is not the full official multimodal checkpoint. The event FAQ explicitly permits community GGUFs in Track 2.

HF revision: 248ec68a63e219e3f061e4c946fc8a20d63b9975. File: 5,059,027,136 bytes. SHA256: a037df8d87ff6caacee794ee85f55342f2152e0d359b7389033300c3bee5f299. Download integrity was verified; the launcher also verifies the file before starting it.

Serving: official llama.cpp b11552 CPU Windows build; 4096-token context, four generation/batch threads, batch 256, microbatch 128 and one slot. Requests use temperature 0, seed 42 and schema-constrained JSON. The complete prompt is in src/model.mjs. Source passages are untrusted data, not instructions. No fine-tuning, training or other runtime model is used. Substantial coding/writing assistance came from Codex; the participant selected the direction and approves consequential external actions.

4. Data and attribution

data/evaluation.json contains six original hand-authored fictional English/Polish letters and expected passage quotes. Cases cover conditional requests, negation, relative dates, a quoted instruction attack and contradictory meeting dates. No actual user letter or personal correspondence was used. Original fictional evaluation content: CC-BY-4.0. Recorded responses are in data/live-evaluation.json; the first development run is retained separately. Data is below 100 MB. No public Hugging Face dataset publication is claimed.

The upstream project template is preserved in track_2b, from commit 09f8e9e2b588db3325869b23c6d5132f5c3756f7. No previously submitted application code is repackaged as this new project. Existing official runtime binaries were reused locally and are excluded from the source package.

5. Evaluation

Twelve technical checks passed: exact offsets, oversize rejection, unsupported/duplicate IDs, malformed/incomplete completions, date literals, local-only endpoint configuration, disagreement flags and actual temporary HTTP origin/host and explicit shutdown checks. Model responses in these tests are injected doubles; the tests do not measure Apertus accuracy.

Separately, six actual local Apertus HTTP analyses ran. Before the independent disagreement rules, action selection produced 6 true positives, 1 false positive and 1 false negative: precision and recall each 6/7. Explicit uncertainty selection produced 4 true positives and no false positives/negatives in this mini-set. The keyword baseline found 7/7 action quotes without extras, but only 2/4 uncertainty quotes. AI did not outperform the baseline on action selection. Observed request latency was 3.62-12.07 seconds, including one cold request.

This is a development set, not a held-out benchmark. The first case informed a prompt revision. Later failures informed heuristic disagreement rules, so no unbiased generalization or production reliability percentage is claimed. The final app flags model-selected no-action phrases and keyword requests omitted by the model. These are separate prompts for human review, not verified classifications. Raw model ID lists remain in the report. Saved six-case metrics predate these rules and are not retroactively presented as improved AI scores.

A larger real Chrome example took 14.7 seconds and also selected a meeting statement as a task. The no-action phrase in that example was flagged separately by the final app. Real source jumps, English translation, edit invalidation and cancellation with no partial result were observed. A file download was attempted but not confirmed; it was replaced with a visible read-only JSON report for manual copying. The final visible JSON report was parsed in Chrome and matched the source hash, four passages and real Apertus identity.

6. Limitations

The model can miss requests and misclassify negation or event descriptions. Exact quotes can still be selected incorrectly or lack sufficient context. Date scanning does not validate calendar dates, resolve relative dates or determine legal deadlines; unsupported formats may be missed. Plain text only: no PDF/OCR parser. The small PL/EN examples do not establish multilingual coverage. Model context limits may cause a visible error. No legal review, adversarial security audit, user study, energy measurement or cloud deployment is claimed.

7. Reproducibility and current status

See docs/RUN.md. Node.js 22.19+ and official llama.cpp b11552 run on the tested Windows computer with about 16 GB RAM. Weights/binaries are excluded from source. The explicit Python setup downloads the pinned model and verifies SHA256. The template environment names LLM_NAME, LLM_BASE_URL and LLM_API_KEY are supported for separately running loopback inference.

Public source: https://github.com/jacman1988-super/quietletter, created from the official template. On 14 October 2026 a standard public Ubuntu 24.04 GitHub runner executed the exact make run from a clean checkout at commit 3f21c5fff7b57497bc21535d20bb02b04a765bd7. It downloaded and verified the weights, built Docker,

started local Apertus, passed all 12 technical tests inside the image and completed a real fictional HTTP review. The selected request, uncertainty quote and literal date matched that example; inference took 28.659 seconds. This is one container smoke test, not a new accuracy benchmark. The first attempt exposed missing make in the build image; that dependency was added before this successful run. No paid runner, uploaded artifact, cache or external model API was used. Verification: https://github.com/jacman1988-super/quietletter/actions/runs/38594216006. Docker remains absent on the author's Windows host.

The standalone CLI accepts one JSON object with text and language pl/en and returns grounded quotes, date literals, raw selections and disagreement flags; one actual native CLI run returned the expected action and date, and its model stopped afterward. Subsequent public documentation-only updates do not change the tested runtime or workflow.

8. Next steps

Record the required under-two-minute demo and obtain explicit final submission/terms approval before entering the contest. Further research should add independent held-out letters with human annotations and improve classification. Preserve attribution and the small development-set limitations; a successful container smoke test does not establish reliable interpretation of correspondence.

License

Application: Apache-2.0. Report/docs and original fictional evaluation content: CC-BY-4.0. Model/runtime retain upstream Apache-2.0/MIT licenses.

References

https://hackapertus.ch/faq#online-hackathon

https://github.com/HackApertus/project-template/tree/main/track_2b

https://huggingface.co/Colby/apertus-v1.5-8b-text-Q4_K_M-GGUF

https://huggingface.co/andreasmartin/apertus-v1.5-8b-text

https://github.com/ggml-org/llama.cpp/releases/tag/b11552

https://hackapertus.ch/terms-and-conditions